

PROJEKT ENGINEERING

Trends bei moderner (agiler) Softwareentwicklung

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg

Beispiele agiler Vorgehensmodelle

Prozessmodell-Topologie

entsprechend verschiedener Gültigkeitsbereiche:

Modell	Gültigkeitsbereich
Unternehmensprozess-Modell	Unternehmenskultur (QM, Teambildung, Arbeitszeit, ...)
Projektprozess-Modell	gesamter Projektentwicklungszyklus
Entwicklungsprozess-Modell	Designen-Implementieren-Testen-Zyklus (D-I-T)

Jedes Modell mit breiterem Gültigkeitsbereich braucht ein geeignetes Modell mit engerem Gültigkeitsbereich !

Prozess-Skalierung

Prozessmodelle passen nie exakt für ein neues Projekt!

-> **Prozess-Skalierung** auf Grund von:

- Produktprofil
- Anwendungsdomäne
- Rahmenbedingungen

Prozess-Skalierung gibt es im agilen Bereich (z.B. Crystal) wie auch im nicht agilen Bereich (z.B. V-Modell-XT)

Nicht nur Prozess anpassen, sondern auch Werkzeuge!

Prozess ist auch **während** eines Projektes anpassbar.

Bild: www.wobst.com

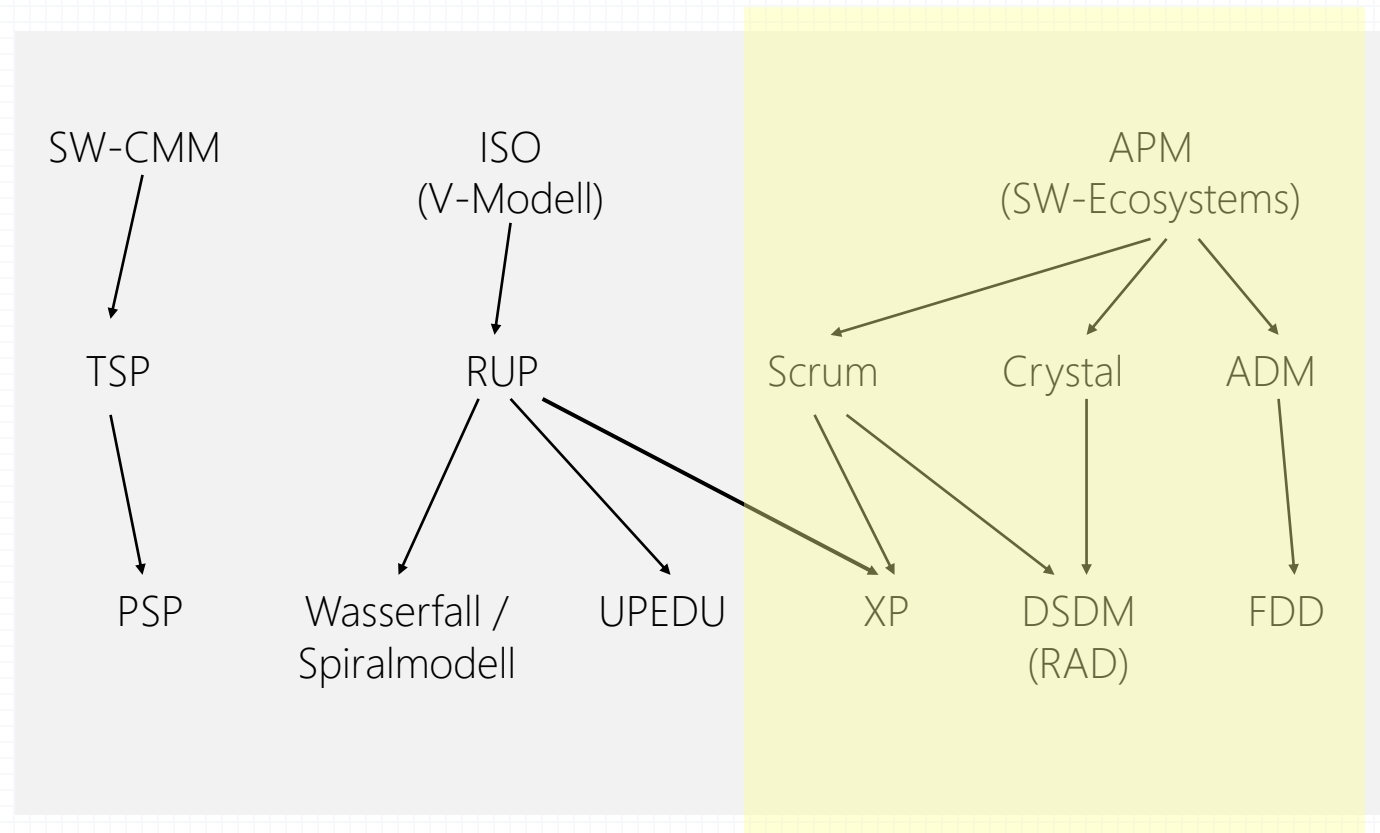


(Agile) Vorgehensmodelle: Beispiele [Wiederholung]

Unternehmensprozess-
Modelle

Projektprozess-
Modelle

Entwicklungsprozess-
Modelle



Die Technik entwickelt sich immer vom Primitiven über das Komplizierte zum Einfachen! (Antoine de Saint Exupery)

Überblick: Agile Vorgehensmodelle

Projektprozess-Modelle:

- ADM (ASD)
- Crystal Family
- Lean Development
- Scrum

Entwicklungsprozess-Modelle:

- DSDM (RAD)
- FDD
- Open Source Development
- XP

Adaptive Software Development (ADM, ASD)

- Kategorie: Projektprozess-Modell
- Guru(s): J. Highsmith, P. Senge („lernende Organisationen“)
- Prinzipien: Anwendung der Theorie komplexer Systeme (z.B. Schwarmverhalten) auf die SW-Entwicklung; Zusammenspiel unabhängiger Agenten führt zu „emergentem Verhalten“ (vgl. Computerpheromonie)
- Praktiken: Spekulieren – Kollaborieren – Lernen
(einfache Regeln mit vielen Beziehungen = komplexes Verhalten, komplexe Regeln mit wenig Beziehungen = dämliches Verhalten)

Crystal Family (Crystal Clear, Yellow, Orange, Red, ...)

Kategorie: Projektprozess-Modell

Guru(s): A. Cockburn

Prinzipien: keine Methode ist für jedes Projekt geeignet
(aus Beobachtungen: „SW-Anthropologie“)

Praktiken: entscheidend ist wie Projektarbeiter miteinander kommunizieren („Information diffundiert“); je nach Organisationskultur, Projektgröße und Kritikalität andere Prozessausprägung (Farbe)
Crystal Family = „Enzyklopädie der Arbeitsergebnisse“

Lean Development

- Kategorie: Projektprozess-Modell
- Guru(s): B. Charette, (M. & T. Poppendieck)
- Prinzipien: „ein Drittel Zeit, ein Drittel Budget, ein Drittel Fehler“, Management und Organisation müssen agil sein bevor es die Entwicklung sein kann; Änderungen bringen Risiken und Chancen: vermeide die einen, nutze die anderen (aus „Lean Production“ entstanden)
- Praktiken: aktive Risikobehandlung zur bestmöglichen Chancenausnutzung, kurze Releasezyklen, Einfachheit kommt vor Automatisierung („Wizards“ ?), „dynamische Stabilität“ (Jongleur), minimal notwendige Funktionalität

Scrum

- Kategorie: Projektprozess-Modell
- Guru(s): K. Schwaber, J. Sutherland, M. Beedle
- Prinzipien: aus der Verfahrenstechnik abgeleitet
(„Softwareentwicklung ist empirischer Prozess“)
- Praktiken: Selbstorganisation, Anpassbarkeit (auch des Prozesses während des Ablaufs), 30-Tage-Sprints mit täglichen kurzen Treffen zur Abstimmung, nach 30 Tagen Abgleich mit (ev. geänderten) Anforderungen

Dynamic Software Development Method (DSDM)

- Kategorie: Entwicklungsprozess-Modell
- Guru(s): (Konsortium, u.a. Oracle, HP, British Airways)
- Prinzipien: aus Rapid Application Development (RAD) heraus entwickelt („Timebox-Idee“), aktive Beteiligung der Anwender, häufige Auslieferung von Produkten, Wertkriterium ist Business Value
- Praktiken: iterativ-inkrementelle Entwicklungen, alle Änderungen umkehrbar (!), Tests sind in Anforderungen integriert, Art der Teamorganisation ist nicht festgelegt, ISO 9000ff.-zertifiziert !

Feature-driven Development (FDD)

Kategorie: Entwicklungsprozess-Modell

Guru(s): J. DeLuca, P. Coad

Prinzipien: kurze Beschreibung (5 Phasen, je 1 Seite, jede umfasst Eintrittskriterien, Aktivitäten, Verifikationsmaßnahmen und Abschlusskriterien)

Praktiken: Entwicklung eines übergreifenden Modells, Feature-Liste mit Feature-Plan, Design und Implementierung pro Feature

Open Source Development

- Kategorie: Entwicklungsprozess-Modell
- Guru(s): R. Stallman (GNU, FSF), L. Torvalds (Linux), ...
- Prinzipien: räumlich verteilte Teams, laufendes Kunden-Feedback, „Irren ist menschlich & kommt häufig vor“
- Praktiken: gemeinsame Codeverantwortung, aber ausgezeichnete Projektverantwortliche (Code Maintainer) mit Letztentscheidung, stark iterativ-inkrementelle Entwicklung inkl. vieler Verzweigungen (Branches), Teilzeitarbeit, viele Debugger pro Entwickler (!),

eXtreme Programming (XP)

Kategorie: Entwicklungsprozess-Modell

Guru(s): K. Beck, R. Jeffries

Prinzipien: Streben nach Einfachheit, unmittelbares Feedback (durch Kunden „vor Ort“), Änderungen sind willkommen

Praktiken: einfaches Design, kurze Inkremente, testgesteuerte Entwicklung, Pair Programming, Refactoring, gemeinsame Verantwortlichkeit für den Code (insgesamt 12)

Beispiele agiler Vorgehensmodelle für größere Unternehmen

(Teils subjektive) Beobachtungen zu agilen Vorgehensmodellen

- Agile Vorgehen werden reifer.
- Agile Vorgehen werden für größere Unternehmen angepasst.
- Agile Vorgehen werden immer träger („fetter“) und dadurch weniger agil.



(Bild: <http://www.theguardian.com>)

Beispiel: Scrum wird immer „fetter“

- „The Return of the **Burndown Chart**“:
 - ursprünglich durch Scrum Board abgelöst
 - auch als „**Burnup-Charts**“ (-> Fortschrittsdiagramm)
- Immer größere Diversifizierung der Rollen:
z.B. bei Gloger
 - **Customer** vs. **User**
 - **Manager**
- Ausweitung auf dislozierte Teams:
 - **Distributed Scrum** (z.B. „Distributed Scrum Primer“:
<http://www.goodagile.com/distributedscrumprimer/DistributedScrumPrimer.pdf>)



(Bild: Idolstore.net)

Agile Vorgehen werden (aus der Not heraus!) breiter einsetzbar

Vorgehensmethoden

- Disciplined Agile [Delivery]
- Scaling Scrum:
 - Scrum of Scrums, Scrum@Scale
 - Spawned Scrum Teams
 - Large-Scale Scrum, Nexus
- Component Teams und Feature Teams (Chapter, Guilds, Squads, Tribes)
- Scaled Agile Framework

Praxisbeispiele

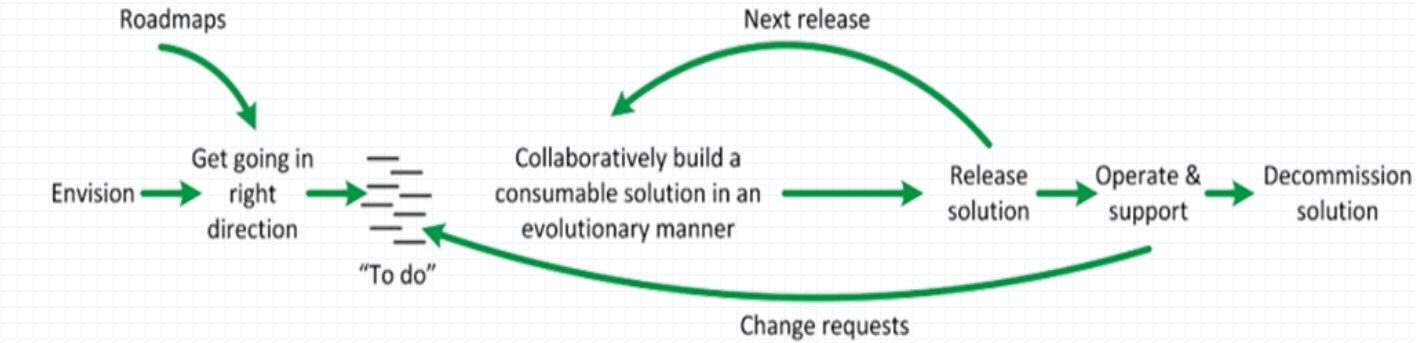
- BMW i-Sparte, Deutschland
- LEGO Digital Solutions, Dänemark
- Intuit, USA

(meistens Kurse, Zertifizierungen etc. -> oft schade ums Geld!)

„Disciplined Agile [Delivery]“ (DA[D]) nach Scott Ambler/Mark Lines

- entwickelt bei IBM Rational (ab 2006); „Toolkit“
- Prozess sehr Scrum-ähnlich
- weitere Rollen:
 - Stakeholder
 - Team Lead (= Agile Coach)
 - Architecture Owner
- Netzwerk aus cross-functional Teams
- betont Risk Management (eigenes Risk Board)

(<https://disciplinedagiledelivery.com>)

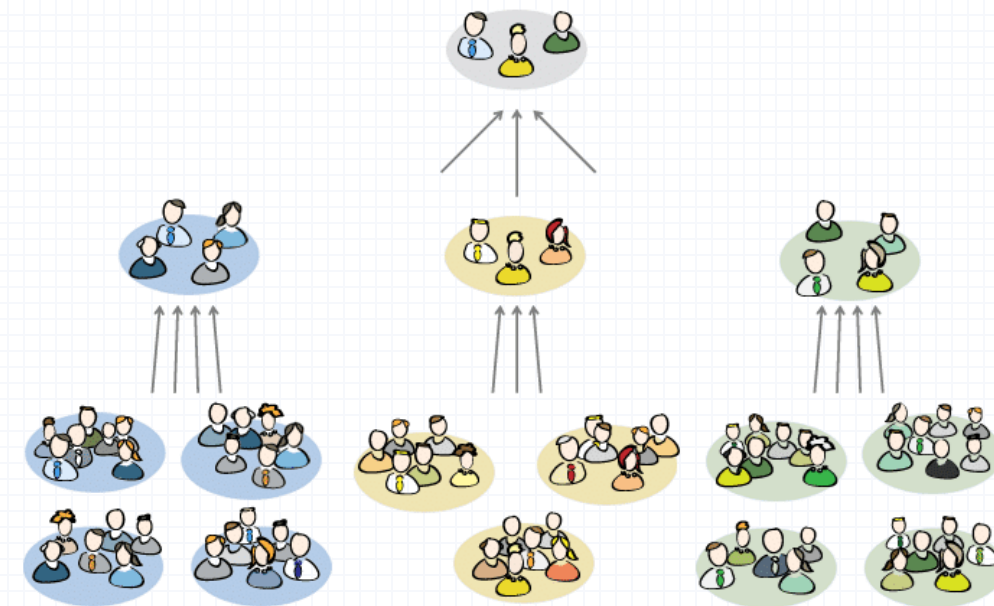


Disciplined Agile
The foundation for business agility



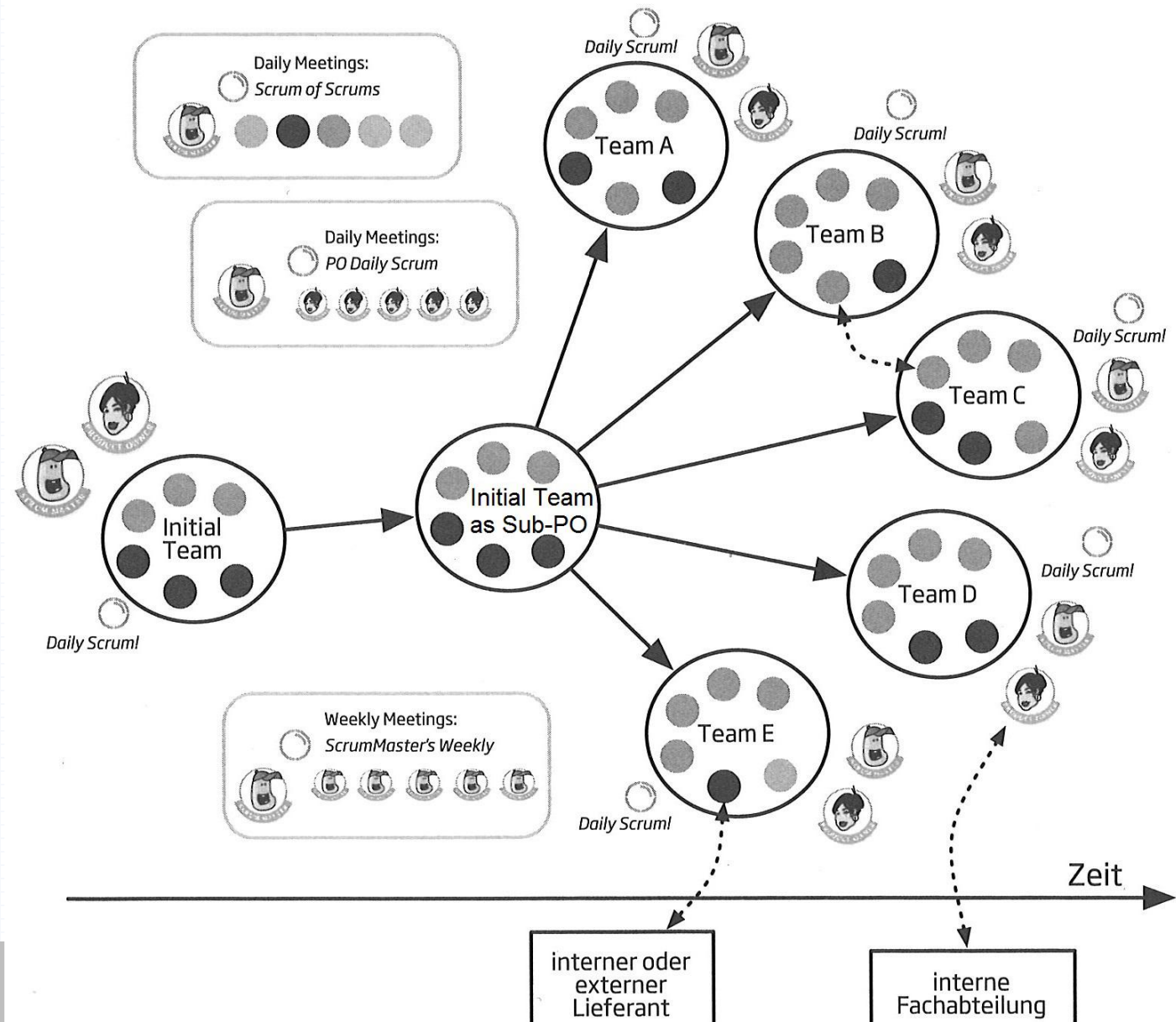
Scaling Scrum: „Scrum of Scrums/Scrum@Scale“ nach Jeff Sutherland

- Konzept von Jeff Sutherland (Scrum-Co-Erfinder) zur Scrum-Skalierung
<http://static1.1.sqspcdn.com/static/f/447037/6486358/1270929593650/Sutherland+200111+proof.pdf?token=Mr0i4TJRikJeMGN0LnufUaybUqw%3D>
- bereits 2001 vorgestellt (auch „meta Scrum“ genannt)
- Zwischenschicht zwischen Scrum Masters, „Experten“ und Unternehmensführung
- tägliche Berichte zu Impediments und Daily Scrum Topics
- skaliert schlecht für komplexe Produkte (keine expliziten PO Meetings!)
-> behoben ab 2006 mit „Scrum@Scale“ (Chief Product Owner – CPO)



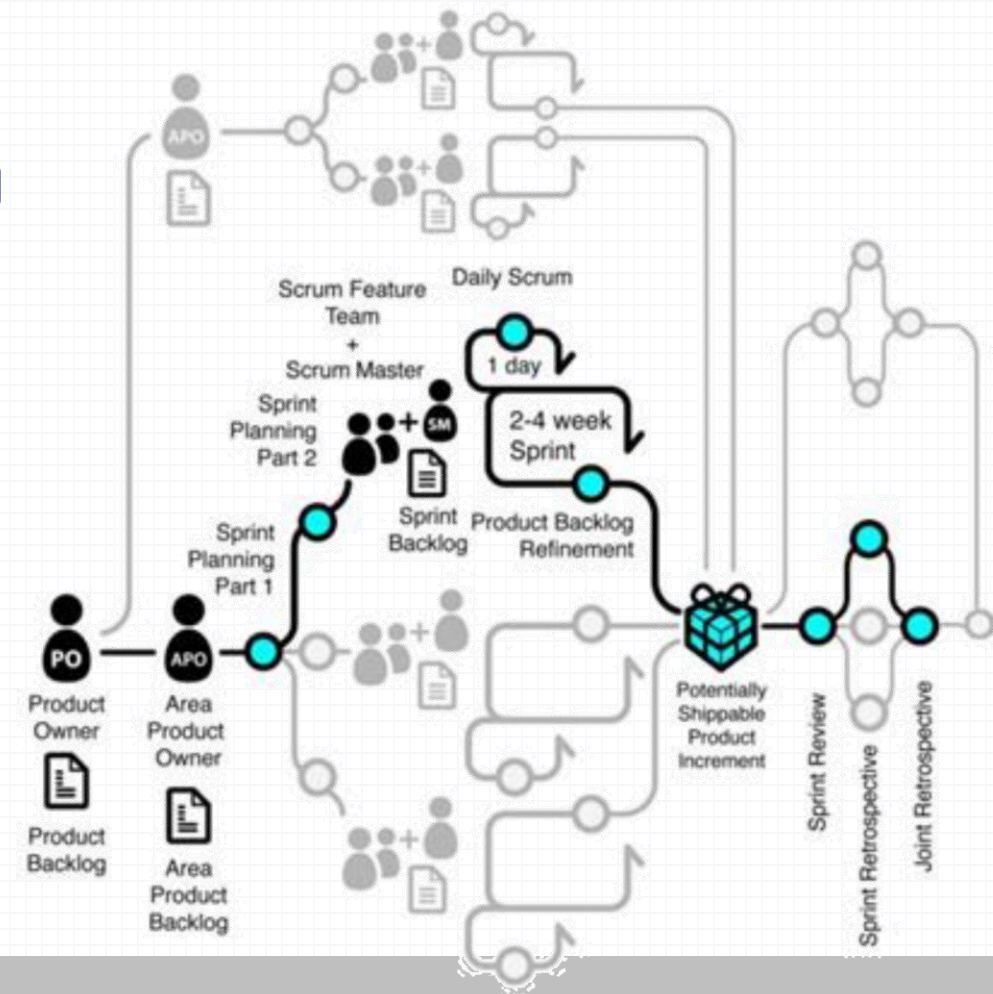
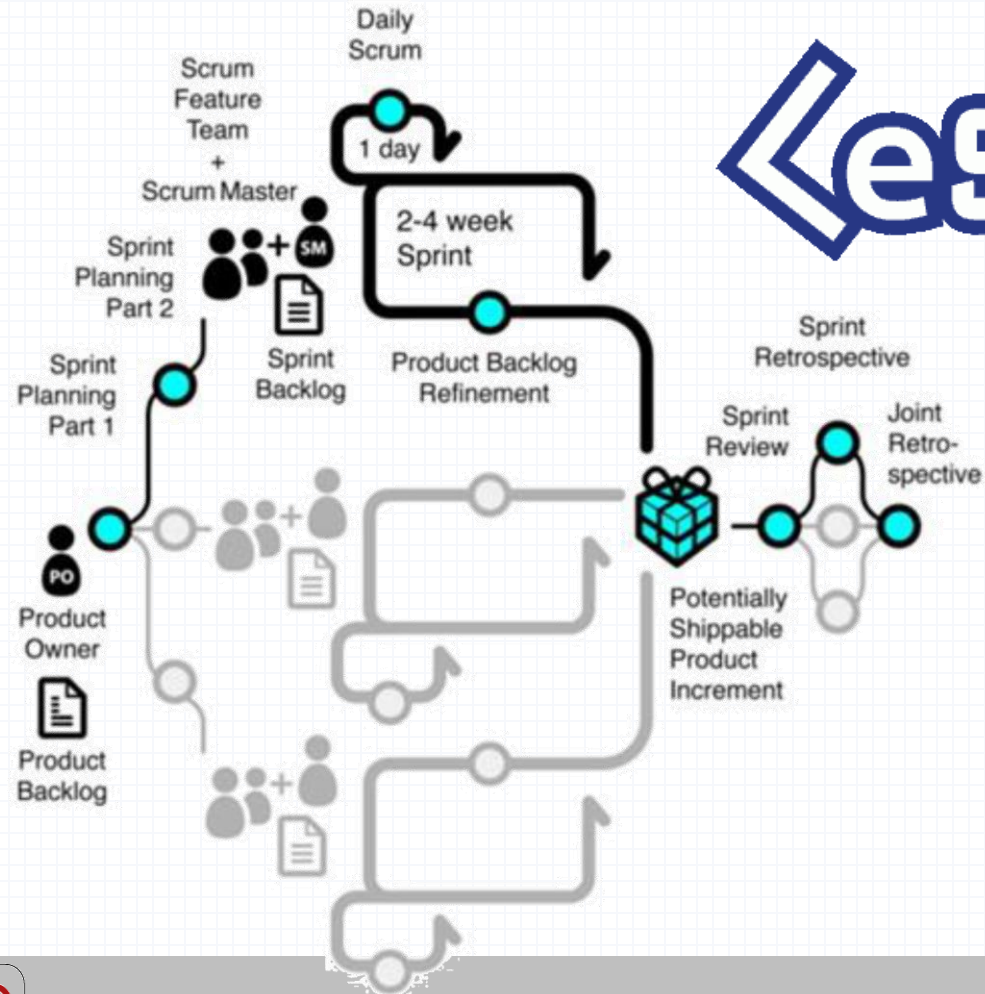
Scaling Scrum: „Spawned Scrum Teams“ nach Boris Gloger

- Ausweitung eines „Initial Team“ auf bis zu zehn kooperierende Teams
- Mitglieder des „Initial Team“ als neue (Sub-)PO!
- ähnlich wie Scrum of Scrums



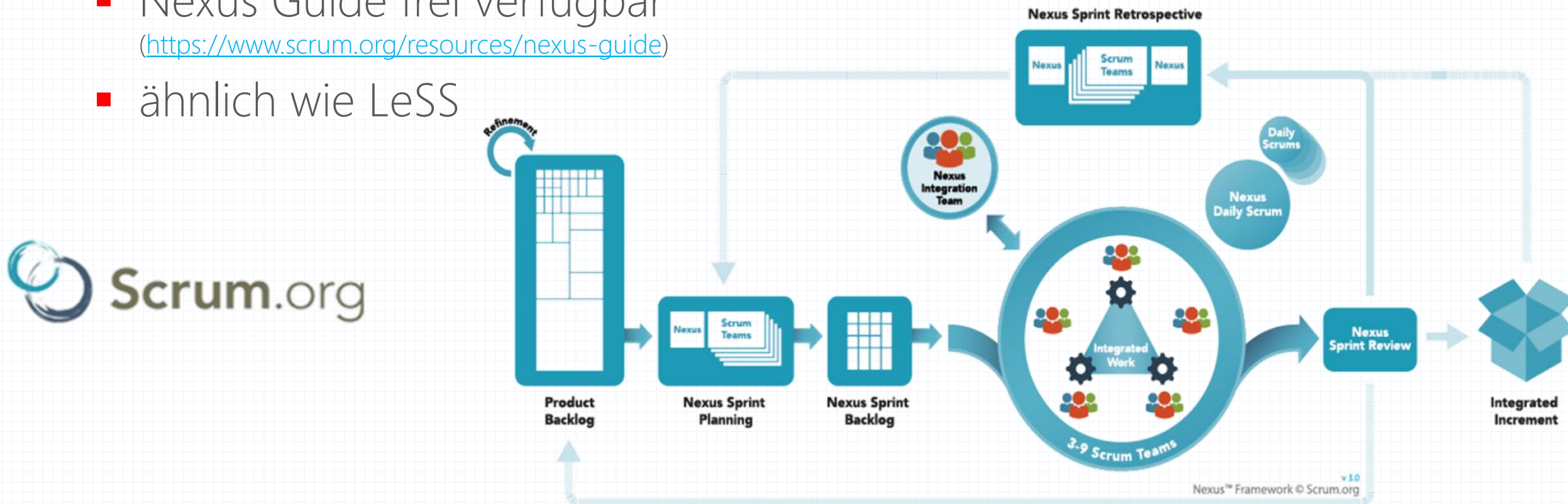
Scaling Scrum: „Large-Scale Scrum“ (LeSS) nach C. Larman/B. Vodde

- Bis zu 10 Teams:
- Mehr als 10 Teams/mehrere Domänen:



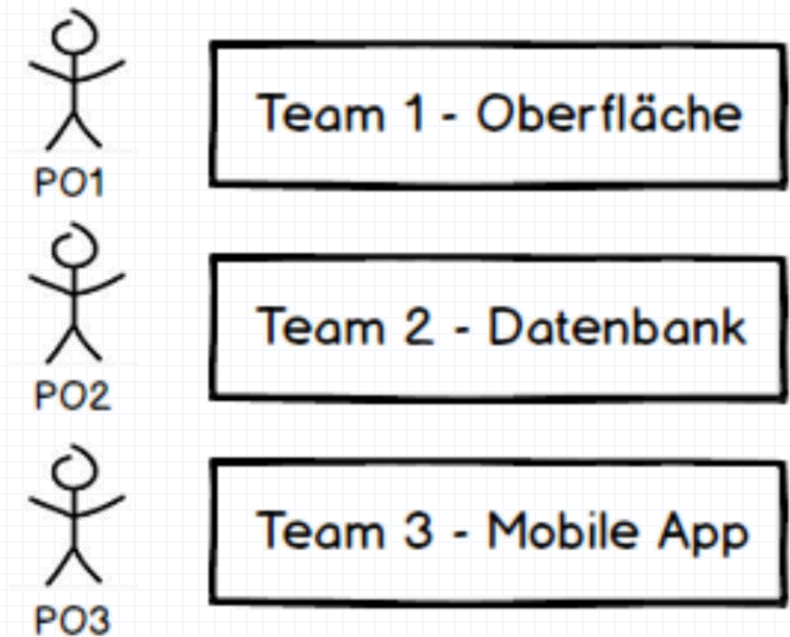
Scaling Scrum: „Nexus“ nach Ken Schwaber (scrum.org)

- Antwort von Ken Schwaber (Scrum-Co-Erfinder) auf andere Scrum-Skalierungsversuche
- Nexus Guide frei verfügbar (<https://www.scrum.org/resources/nexus-guide>)
- ähnlich wie LeSS

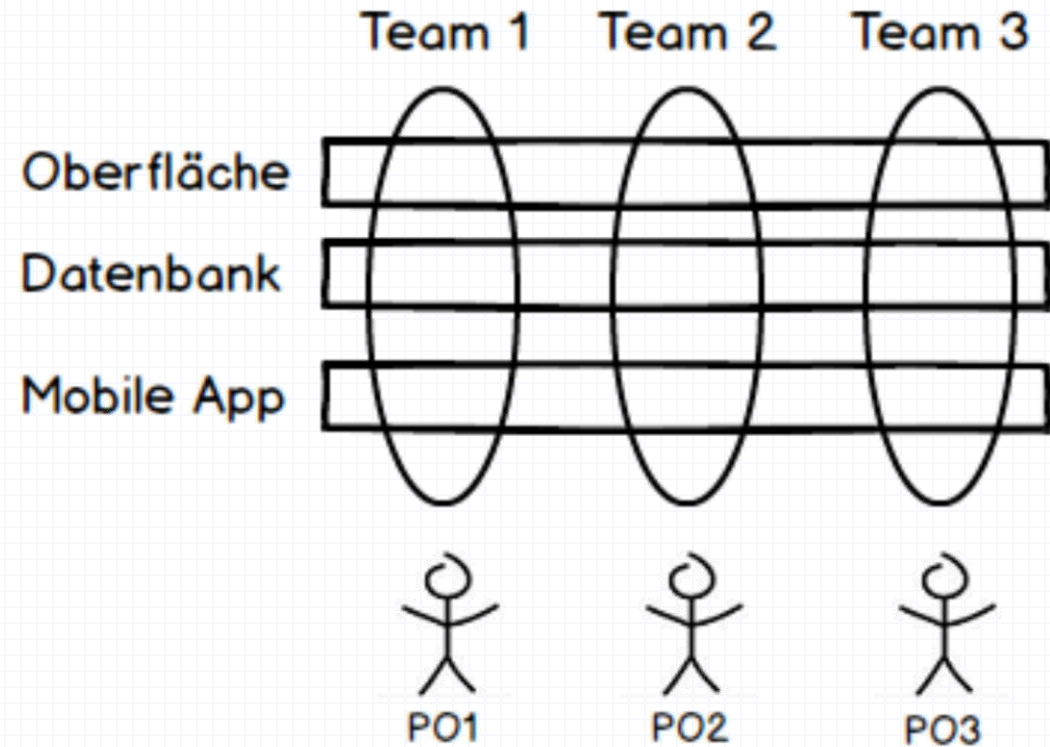


„Component Teams“ und „Feature Teams“

Component Teams



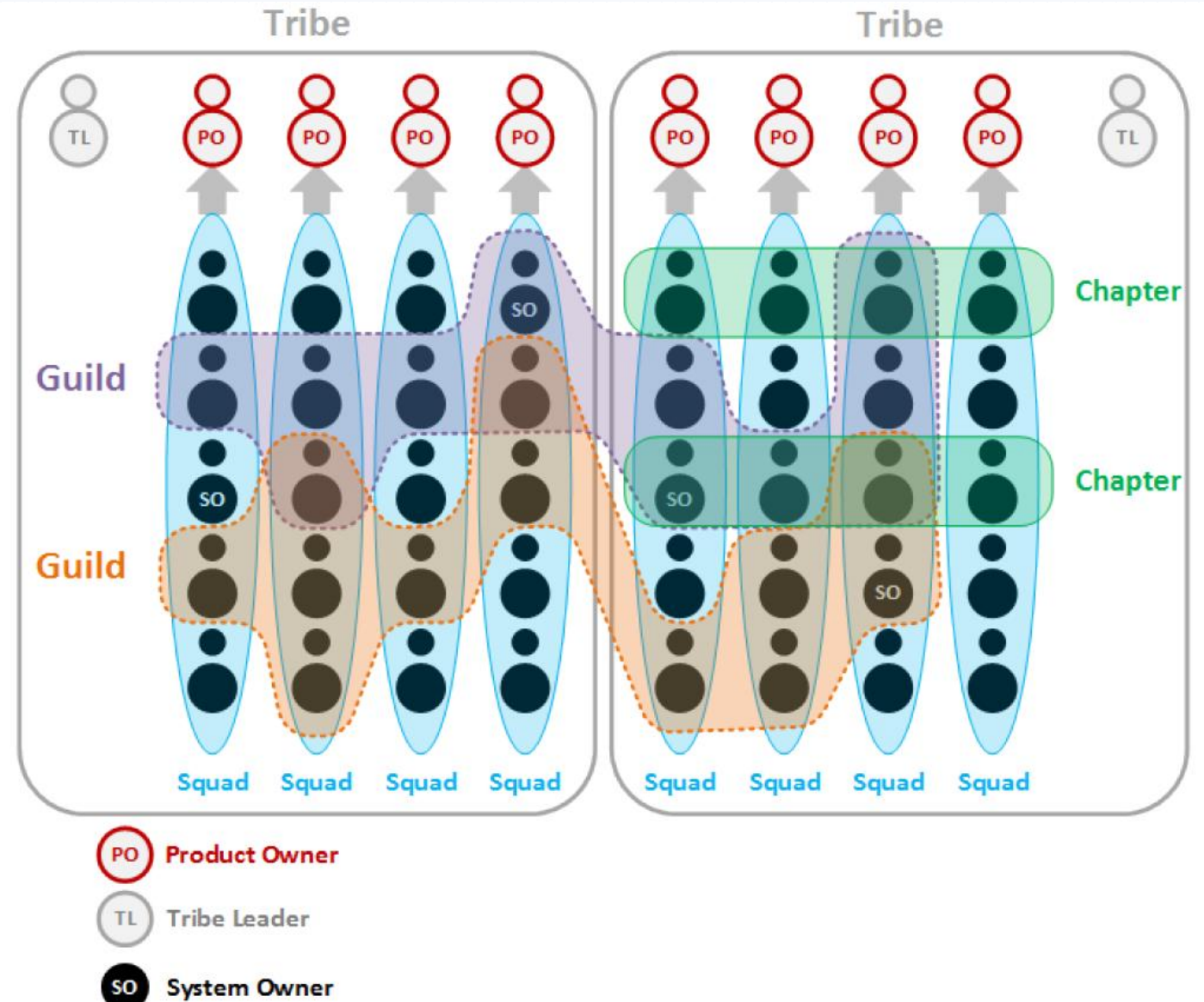
Feature Teams



Einsatz von Component Teams und Feature Teams bei Spotify

Gruppierung in:

- **Chapter**
(Components je Tribe)
- **Guilds**
(Components über mehrere Tribes)
- **Squads**
(Features)
- **Tribes**
(Feature-Gruppen)



„Scaled Agile Framework“ (SAFe) nach Dean Leffingwell



- und wieder einmal ein Buch (2007)...
- integriert Produktmanagementzyklus (nach Don Reinersten)
- fokussiert eine „Big Picture View“ aus Produktsicht -> bekannte (IT-)Governance-Techniken
- längerfristige Roadmap
- Product Manager als Leiter der PO
- „hardening iteration“ zur Produktstabilisierung
- kritisiert als „zu starr, nicht mehr agil“



(z.B. <https://blog.seibert-media.net/blog/2021/04/13/scaled-agile-framework-ein-nuetzliches-modell-und-schlechte-adaptionen-interview-mit-sylvius-gerber>)



Praxisbeispiel 1: BMW i-Sparte, Deutschland (ab 2012)

Situation

- Unified Sales Platform (für i-Modelle)
- rasche, fixe Go-Live-Termine
- sehr kurze Time-to-Market für neue Features (Wochen!)
- rasches Mitarbeiterwachstum

Lösung

- teamübergreifende **Design Governance Group** - DCG
- teamübergreifende **Test Management Group** - TMG
- Strukturierung in Feature Teams (**End-to-End-Funktionalität**)
- **Lead Developer**, mehreren Teams übergeordnet

Erkenntnisse

- EIN Product Backlog für Vision entscheidend
- Product Backlog Refinements wichtig (Sprintmitte, DCG und TMG dabei)



Praxisbeispiel 2: LEGO Digital Solutions, Dänemark (ab 2014)



Situation

- Wachstum von 5 auf 15-20 Teams in 1 Jahr
- Flaschenhals bei PO

Lösung

- **Big Room Planning** alle 8 Wochen
- je 8 Wochen 3 Sprints + Innovation Sprint
- eine Art Kick-off mit allen (!) Stakeholdern -> „Big Room“
- Basis sind alle Team Backlogs + übergreifender Program Backlog
- Review-Ergebnisse -> adaptierte Vision -> adaptierte Roadmap
- **Team Breakouts:** pull-basierte Übernahme der Arbeitspakete durch Teams



Bild: www.businesswire.com)

Erkenntnisse

- Management, Mitarbeiter und Stakeholder auf gleicher Ebene eingebunden
-> ziehen am selben Strang!
- Mitarbeiter müssen positiv eingestellt sein (kritische Masse davon)
-> **Agile Culture** wichtig

Praxisbeispiel 3: Intuit, USA (ab 2016)

Situation

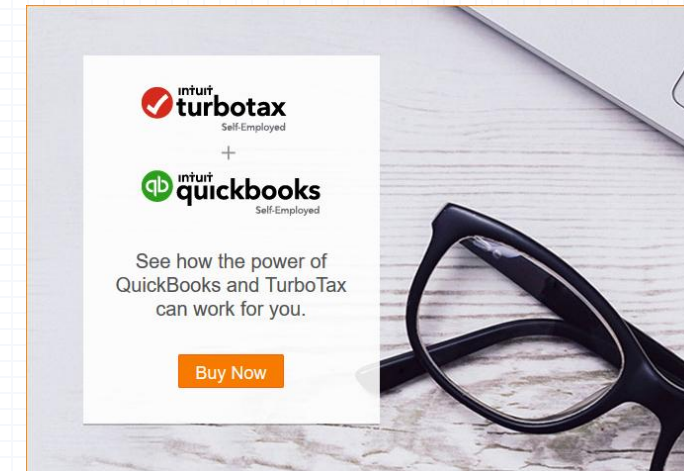
- (IT-)Innovationen vielfach nur durch kleine Unternehmen/Start-ups
- verlieren Innovationskraft bzw. Innovatoren durch Wachstum

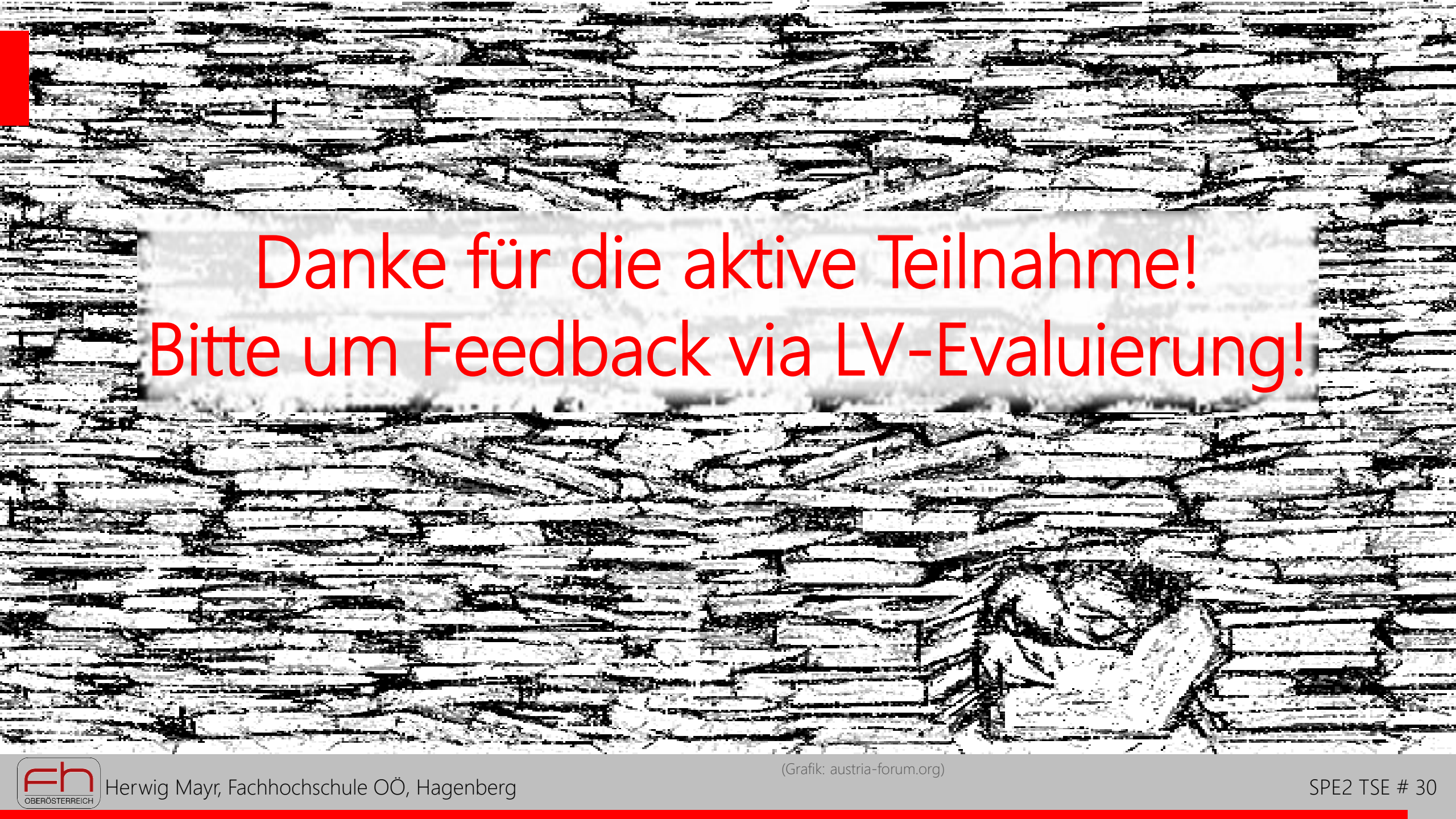
Lösung

- Innovations- und Experimentierkultur
- minimales Produkt auf Basis einer Idee

Erkenntnisse

- rasches Kundenfeedback entscheidend
- „fail fast“ ist wichtig
- Hypothesen statt Anforderungen: **Hypotheses-Driven Development**
(eigentl. "alter Hut": Stefan H. Tomke – Experimentation Matters, Harvard B.R. Press 2003)





Danke für die aktive Teilnahme!
Bitte um Feedback via LV-Evaluierung!

PROJEKT ENGINEERING

Trends bei moderner (agiler) Softwareentwicklung

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg